Community Discussion Platform

19 March 2026

Team Members

1. 241CS205, Akhil Dang, akd.241cs205@nitk.edu.in, 7505369839
2. 241CS247, Raju Kumar, raju.241cs247@nitk.edu.in, 9608392121
3. 241CS250, S Ashwin, sashwin.241cs250@nitk.edu.in, 8088437882
4. 241CS252, Samarth Talawar, samarthpt216@gmail.com, 8217215496

Contents

1 Introduction **2**

2 Literature Survey **2**

3 Objectives **2**

4 Features **2**

5 Requirement Specification **3**
- 5.1 Frontend . . . 3
- 5.2 Backend . . . 3
- 5.3 Database . . . 3

6 System Design **3**
- 6.1 Database Schema Design . . . 3
- 6.2 ER Diagram . . . 6
- 6.3 Tables . . . 7

7 Implementation **8**
- 7.1 Tools and Technologies Used . . . 8

8 References **8**

1 Introduction

The Community Discussion Platform is a pseudo-anonymous web application designed for campus environments to promote open and free discussions among students. Inspired by platforms like Reddit, this system enables users to create posts, participate in discussions, and share opinions without revealing their real identity.

The platform ensures a balance between anonymity and accountability, making it suitable for academic environments where students may hesitate to express their views openly.

2 Literature Survey

Several online discussion platforms such as Reddit, Quora, and Stack Overflow have demonstrated the importance of community-driven content sharing. Reddit, in particular, provides a pseudo-anonymous structure that encourages free speech while maintaining moderation.

Existing campus platforms often lack anonymity, leading to reduced participation. This project addresses that gap by introducing a structured, database-driven system that ensures efficient data handling, scalability, and user engagement.

3 Objectives

- To design a scalable database for handling user-generated content.
- To implement pseudo-anonymous posting.
- To enable structured discussions through comments.
- To ensure efficient data retrieval and storage.
- To promote freedom of speech within campus communities.

4 Features

- User authentication and session management
- Pseudo-anonymous identity system
- Create, edit, and delete posts
- Comment on posts
- Upvote/Downvote system
- Organized discussion threads
- Responsive UI for accessibility

5 Requirement Specification

5.1 Frontend

- React (with Vite)
- Tailwind CSS for styling
- Browser (Chrome/Firefox recommended)

5.2 Backend

- Node.js (v18+ recommended)
- Express.js framework
- RESTful API design

5.3 Database

- MySQL Server

6 System Design

6.1 Database Schema Design

The database is designed to efficiently manage users, posts, comments, and interactions. Relationships ensure normalization and minimize redundancy.

```sql
-- USERS TABLE
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- COMMUNITIES TABLE
CREATE TABLE Communities (
    community_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    description TEXT,
    created_by INT,
    FOREIGN KEY (created_by) REFERENCES Users(user_id)
        ON DELETE SET NULL
);

-- COMMUNITY MEMBERS TABLE
```

```sql
CREATE TABLE Community_Members (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    community_id INT,
    joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
        ON DELETE CASCADE,
    FOREIGN KEY (community_id) REFERENCES Communities(
        community_id)
        ON DELETE CASCADE
);

-- MODERATORS TABLE
CREATE TABLE Moderators (
    user_id INT,
    community_id INT,
    joined_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, community_id),
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
        ON DELETE CASCADE,
    FOREIGN KEY (community_id) REFERENCES Communities(
        community_id)
        ON DELETE CASCADE
);

-- POSTS TABLE
CREATE TABLE Posts (
    post_id INT AUTO_INCREMENT PRIMARY KEY,
    community_id INT,
    user_id INT,
    title VARCHAR(255) NOT NULL,
    content TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (community_id) REFERENCES Communities(
        community_id)
        ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
        ON DELETE SET NULL
);

-- COMMENTS TABLE
CREATE TABLE Comments (
    comment_id INT AUTO_INCREMENT PRIMARY KEY,
    post_id INT,
    user_id INT,
    content TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (post_id) REFERENCES Posts(post_id)
        ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
```

```sql
        ON DELETE SET NULL
);

-- VOTES TABLE
CREATE TABLE Votes (
    vote_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
    post_id INT,
    vote_type ENUM('UPVOTE', 'DOWNVOTE') NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(user_id, post_id),
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
        ON DELETE CASCADE,
    FOREIGN KEY (post_id) REFERENCES Posts(post_id)
        ON DELETE CASCADE
);

-- TAGS TABLE
CREATE TABLE Tags (
    tag_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) UNIQUE NOT NULL
);

-- POST TAGS (M:N RELATION)
CREATE TABLE Post_Tags (
    post_id INT,
    tag_id INT,
    PRIMARY KEY (post_id, tag_id),
    FOREIGN KEY (post_id) REFERENCES Posts(post_id)
        ON DELETE CASCADE,
    FOREIGN KEY (tag_id) REFERENCES Tags(tag_id)
        ON DELETE CASCADE
);

-- SAVED POSTS TABLE
CREATE TABLE Saved_Posts (
    user_id INT,
    post_id INT,
    saved_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, post_id),
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
        ON DELETE CASCADE,
    FOREIGN KEY (post_id) REFERENCES Posts(post_id)
        ON DELETE CASCADE
);

-- REPORTS TABLE
CREATE TABLE Reports (
    report_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT,
```

```sql
    post_id INT,
    reason TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
        ON DELETE CASCADE,
    FOREIGN KEY (post_id) REFERENCES Posts(post_id)
        ON DELETE CASCADE
);
```

6.2 ER Diagram

Figure 1 clearly shows the ER Diagram for the following Tables.

- **Entities:** User, Community, Post, Comment, Vote, Tag, Report, Saved_Post, Community_Member, Moderator
- **Relationships:**
  - A User creates Communities
  - A User joins Communities (via Community_Members)
  - A Community has many Posts
  - A User creates Posts within a Community
  - A User writes Comments on Posts
  - A Post has many Comments
  - A User votes on Posts (via Votes)
  - A Post receives Votes
  - A Post can have multiple Tags (via Post_Tags)
  - A User can save Posts (via Saved_Posts)
  - A User can report Posts (via Reports)
  - A User can be a Moderator of a Community

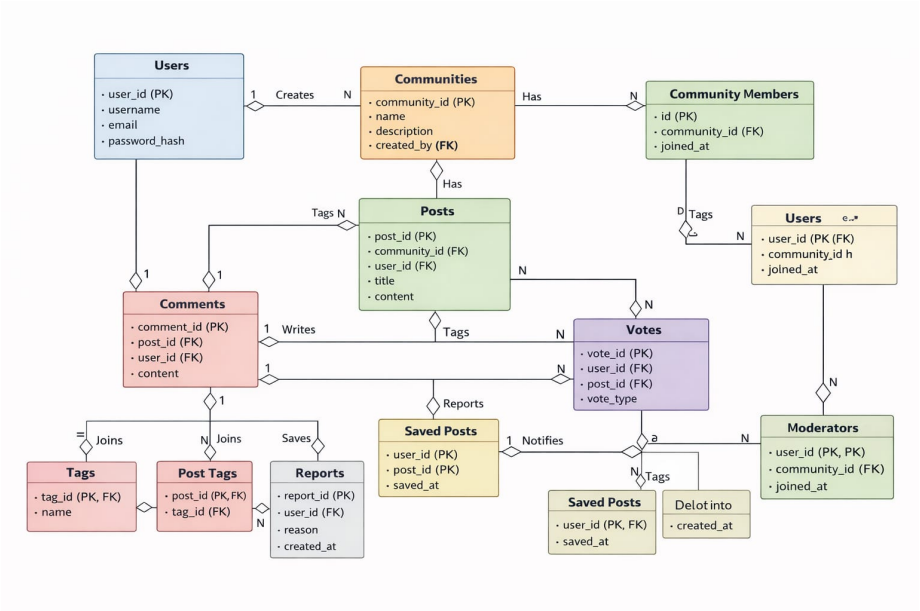

Figure 1: ER Diagram

6.3 Tables

- **Users**: user_id, username, email, password_hash, created_at
- **Communities**: community_id, name, description, created_by
- **Community_Members**: id, user_id, community_id, joined_at
- **Moderators**: user_id, community_id, joined_at
- **Posts**: post_id, community_id, user_id, title, content, created_at
- **Comments**: comment_id, post_id, user_id, content, created_at
- **Votes**: vote_id, user_id, post_id, vote_type, created_at
- **Tags**: tag_id, name
- **Post_Tags**: post_id, tag_id
- **Saved_Posts**: user_id, post_id, saved_at
- **Reports**: report_id, user_id, post_id, reason, created_at

7 Implementation

7.1 Tools and Technologies Used

Frontend:

- React + Vite for fast development
- Tailwind CSS for responsive UI design

Backend:

- Node.js for server-side execution
- Express.js for API handling

Database:

- MySQL for relational data storage

Project Structure:

```
community-platform/

 client/         # React frontend
 server/         # Express backend
 database/       # SQL schema
 README.md
```

8 References

- Reddit - https://www.reddit.com
- Node.js Documentation - https://nodejs.org
- React Documentation - https://react.dev
- MySQL Documentation - https://dev.mysql.com/doc/